ΜΑΘΗΜΑ 5 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.2

Αρχεία και εξαιρέσεις

Τα δεδομένα μένουν — και τα λάθη δεν ρίχνουν το πρόγραμμα

Θεωρία + 2 ασκήσεις + 3 προγράμματα με λύσεις Διάρκεια: 4 ώρες (3η–6η) 23 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 2/11/2026

Λαμπρόπουλος Χρήστος

Τι θα ξέρεις στο τέλος

1. Να **γράφεις** και να **διαβάζεις** αρχεία κειμένου με **with open(...)**.
2. Να ξεχωρίζεις τους τρόπους **r**, **w**, **a** — και να μη σβήνεις κατά λάθος ένα αρχείο.
3. Να δουλεύεις αρχεία **CSV** με το module **csv** (ανοίγουν και στο Excel).
4. Να αποθηκεύεις ολόκληρες δομές σε **δυαδικό** αρχείο με το **pickle**.
5. Να «πιάνεις» λάθη με **try / except / else / finally** αντί να σταματά το πρόγραμμα.
6. Να φτιάχνεις **τηλεφωνικό κατάλογο** που θυμάται τις επαφές και μετά το κλείσιμο.

Πώς είναι οργανωμένο το 4ωρο

Ώρες 1–2: αρχεία (κείμενο, CSV, δυαδικά) και εξαιρέσεις με live coding. **Ώρες 3–4:** εργαστήριο — ασφαλής είσοδος βαθμών, στατιστικά από CSV και στο τέλος ο **κατάλογος της προηγούμενης φοράς, με αποθήκευση**.

ΤΟ ΑΡΧΕΊΟ Στον φάκελο του μαθήματος υπάρχει έτοιμο το **grades.csv** για το Εργαστήριο 2. Αντίγραψέ το δίπλα στο .py σου.

Θυμάσαι από την προηγούμενη φορά;

ΕΚΦΏΝΗΣΗ

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Τι δίνει κάθε έκφραση;

ΛΎΣΗ

ΈΚΦΡΑΣΗ	ΑΠΟΤΈΛΕΣΜΑ	ΓΙΑΤΊ
`len({1, 1, 2})`	`2`	σύνολο: μοναδικά
`{"a": 1}["a"]`	`1`	τιμή του κλειδιού
`{"a": 1}.get("b", 0)`	`0`	ασφαλής πρόσβαση
`{"a": 1}["b"]`	**KeyError**	το κλειδί λείπει
`int("πέντε")`	**ValueError**	δεν είναι αριθμός

Τα δύο σφάλματα

Το **KeyError** και το **ValueError** μέχρι τώρα **σταματούσαν** το πρόγραμμα. Σήμερα θα μάθουμε να τα «πιάνουμε».

ΣΎΝΔΕΣΗ Ο τηλεφωνικός κατάλογος της προηγούμενης φοράς είναι το «πειραματόζωό» μας σήμερα.

Δύο αδυναμίες του καταλόγου μας

1 · ΚΛΕΊΝΕΙ ΤΟ ΠΡΌΓΡΑΜΜΑ, ΧΆΝΟΝΤΑΙ ΤΑ ΠΆΝΤΑ

```
phone = {}
phone["Άννα"] = "6971111111"
# ... κλείνουμε το πρόγραμμα ...
# ξανανοίγουμε:
print(phone)        # {}
```

2 · ΈΝΑ ΛΆΘΟΣ ΤΟΥ ΧΡΉΣΤΗ, ΤΟ ΡΊΧΝΕΙ

```
age = int(input("Ηλικία: "))
Ηλικία: είκοσι
Traceback (most recent call last):
  ...
ValueError: invalid literal for int()
```

Η λύση: αρχεία

Η μεταβλητή ζει στη **μνήμη** (RAM) και σβήνει. Το αρχείο ζει στον **δίσκο** και μένει.

Η λύση: εξαιρέσεις

Με **try / except** λες: «δοκίμασε· αν στραβώσει, κάνε αυτό» — αντί να σταματήσει το πρόγραμμα.

Το σημερινό σχέδιο

Ώρες 1–2: αρχεία, μετά εξαιρέσεις. Ώρες 3–4: τα ενώνουμε σε έναν κατάλογο που **θυμάται** και **αντέχει** τα λάθη.

ΓΙΑΤΊ ΜΑΖΊ Με τα αρχεία έρχονται και τα πρώτα «πραγματικά» λάθη: αρχείο που δεν υπάρχει, χαλασμένα δεδομένα. Γι' αυτό τα μαθαίνουμε μαζί.

open() και with

```
# ο κλασικός τρόπος
f = open("notes.txt", "w", encoding="utf-8")
f.write("Γεια\n")
f.close()

# ο σωστός τρόπος
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Γεια\n")
```

MODE	ΤΙ ΚΆΝΕΙ
`"r"`	ανάγνωση (προεπιλογή) · **σφάλμα** αν δεν υπάρχει
`"w"`	εγγραφή · **σβήνει** ό,τι είχε το αρχείο
`"a"`	προσθήκη **στο τέλος** · κρατά τα παλιά
`"rb"` `"wb"`	ανάγνωση / εγγραφή **δυαδικού** αρχείου

Γιατί with

Κλείνει το αρχείο **μόνο του** μόλις τελειώσει το μπλοκ — ακόμα κι αν γίνει λάθος στη μέση. Με open/close είναι εύκολο να ξεχάσεις το close().

encoding="utf-8"

Γράφε το **πάντα**. Χωρίς αυτό, τα **ελληνικά** μπορεί να βγουν «κινέζικα» στα Windows.

Πού φτιάχνεται το αρχείο

Στον φάκελο από όπου **τρέχει** το πρόγραμμα. Στο VS Code: **File → Open Folder** στον φάκελο της άσκησης.

ΠΡΟΣΟΧΉ Το **"w"** σε αρχείο που υπάρχει το **αδειάζει αμέσως**, πριν γράψεις οτιδήποτε. Δεν υπάρχει αναίρεση.

Γράφουμε σε αρχείο

```
with open("shopping.txt", "w", encoding="utf-8") as f:
    f.write("ψωμί\n")
    f.write("γάλα\n")
    print("αυγά", file=f)

with open("shopping.txt", "a", encoding="utf-8") as f:
    f.write("τυρί\n")
```

write() ή print(file=f);

Το **write** γράφει **ακριβώς** ό,τι του δώσεις — την αλλαγή γραμμής **\n** τη βάζεις εσύ. Το **print(..., file=f)** τη βάζει μόνο του.

ΤΙ ΈΧΕΙ ΤΟ SHOPPING.TXT

```
ψωμί
γάλα
αυγά
τυρί
```

w, μετά a

Το πρώτο μπλοκ **φτιάχνει** το αρχείο από την αρχή. Το δεύτερο **προσθέτει** μια γραμμή στο τέλος, χωρίς να σβήσει τις τρεις πρώτες.

Το write θέλει κείμενο

Το **f.write(17)** δίνει **TypeError**. Γράφεις **f.write(str(17))** ή **f.write(f"{17}\n")** — ή απλώς **print(17, file=f)**.

ΔΟΚΊΜΑΣΕ Άνοιξε το shopping.txt στο VS Code ή στο Σημειωματάριο μετά την εκτέλεση. Είναι ένα συνηθισμένο αρχείο κειμένου.

Διαβάζουμε από αρχείο

1 · ΟΛΌΚΛΗΡΟ ΜΑΖΊ

```
with open("shopping.txt", encoding="utf-8") as f:
    content = f.read()
print(content)
print(len(content.split("\n")))
```

2 · ΓΡΑΜΜΉ-ΓΡΑΜΜΉ

```
with open("shopping.txt", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

ΜΈΘΟΔΟΣ	ΤΙ ΔΊΝΕΙ
`f.read()`	όλο το αρχείο σε **ένα** κείμενο
`f.readline()`	την **επόμενη** γραμμή
`f.readlines()`	**λίστα** με όλες τις γραμμές
`for line in f:`	μία γραμμή σε κάθε γύρο — η πιο συνηθισμένη

Γιατί strip()

Κάθε γραμμή τελειώνει με **\n**. Το print βάζει κι αυτό ένα **\n**, οπότε βλέπεις κενές γραμμές ανάμεσα. Το **strip()** το κόβει.

Όλα είναι κείμενο

Ό,τι διαβάζεις από αρχείο είναι **str** — ακριβώς όπως με την **input()**. Για πράξεις: **int()** ή **float()**.

ΧΩΡΊΣ MODE Όταν δεν γράφεις mode, η open() ανοίγει για **ανάγνωση** ("r"). Αν το αρχείο δεν υπάρχει: **FileNotFoundError**.

Τι έχει τελικά το αρχείο;

ΕΚΦΏΝΗΣΗ

Τα τέσσερα μπλοκ εκτελούνται με τη σειρά. Τι περιέχει το log.txt στο τέλος; Πόσες γραμμές έχει;

```
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("A\n")
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("B\n")
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("Γ\n")
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("Δ")
```

Οι δύο παγίδες

Το τρίτο **"w"** σβήνει τα πάντα. Και το **"Δ"** γράφτηκε χωρίς \n — ό,τι προστεθεί μετά θα κολλήσει στην ίδια γραμμή.

ΛΎΣΗ

ΜΠΛΟΚ	MODE	ΤΟ ΑΡΧΕΊΟ ΜΕΤΆ
1	"w"	A
2	"a"	A · B
3	"w"	Γ (το A και το B σβήστηκαν)
4	"a"	Γ · Δ

Απάντηση

Γ και **Δ** — δύο γραμμές.

Ο κανόνας

Θέλεις να **κρατήσεις** τα παλιά; → **"a"**. Θέλεις **καθαρό** αρχείο; → **"w"**.

ΣΤΗΝ ΠΡΆΞΗ Τα αρχεία καταγραφής (logs) ανοίγουν πάντα με "a": κάθε εκτέλεση προσθέτει τις γραμμές της στο τέλος.

Αριθμοί από αρχείο

NUMBERS.TXT

```
12
7

30
```

```
total = 0
with open("numbers.txt", encoding="utf-8") as f:
    for line in f:
        line = line.strip()
        if line == "":
            continue          # κενή γραμμή: την προσπερνάμε
        total += int(line)
print("Άθροισμα:", total)
Άθροισμα: 49
```

Βήμα 1: strip()

Το "12\n" γίνεται "12". Το int() δέχεται και κενά, αλλά η **κενή γραμμή** όχι.

Βήμα 2: έλεγχος

Η κενή γραμμή γίνεται "" → το **continue** την προσπερνά.

Βήμα 3: int()

Μόνο τότε μετατρέπουμε και **αθροίζουμε**.

Τι θα γινόταν χωρίς τον έλεγχο

int("") → **ValueError** στην 3η γραμμή. Θα δούμε σε λίγο πώς το «πιάνουμε» και με try/except.

ΜΟΤΊΒΟ Άνοιξε → για κάθε γραμμή: καθάρισε, έλεγξε, μετάτρεψε, χρησιμοποίησε. Έτσι διαβάζονται σχεδόν όλα τα αρχεία δεδομένων.

Αρχεία CSV: πίνακας σε κείμενο

```
import csv

rows = [["Άννα", 17], ["Νίκος", 9]]

with open("grades.csv", "w", newline="",
          encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "grade"])
    writer.writerows(rows)
```

ΜΈΘΟΔΟΣ	ΓΡΆΦΕΙ
writer.writerow(list)	**μία** γραμμή
writer.writerows(lists)	**πολλές** γραμμές μαζί

Γιατί newline=""

Στα Windows, χωρίς αυτό, το csv γράφει μια **κενή γραμμή** ανάμεσα σε κάθε εγγραφή.

ΤΙ ΈΧΕΙ ΤΟ GRADES.CSV

```
name,grade
Άννα,17
Νίκος,9
```

Τι είναι το CSV

Comma-Separated Values: κάθε γραμμή μια εγγραφή, οι τιμές χωρισμένες με **κόμμα**. Ανοίγει στο Excel σαν πίνακας.

Ελληνικά στο Excel

Αν το Excel δείχνει «κινέζικα», γράψε **encoding="utf-8-sig"**. Βάζει ένα «σήμα» στην αρχή που λέει στο Excel ότι είναι UTF-8.

ΓΙΑΤΊ ΌΧΙ ΑΠΛΆ WRITE Αν μια τιμή περιέχει κόμμα («Αθήνα, Ελλάδα»), το csv την κλείνει μόνο του σε εισαγωγικά. Με write θα «έσπαγε» η γραμμή.

Διαβάζουμε CSV

1 · CSV.READER · ΚΆΘΕ ΓΡΑΜΜΉ ΛΊΣΤΑ

```
import csv
with open("grades.csv", newline="",
          encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)   # η 1η γραμμή
    for row in reader:
        print(row[0], int(row[1]))
Άννα 17  ·  Νίκος 9
```

2 · CSV.DICTREADER · ΚΆΘΕ ΓΡΑΜΜΉ ΛΕΞΙΚΌ

```
import csv
with open("grades.csv", newline="",
          encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["grade"])


Άννα 17  ·  Νίκος 9
```

next(reader)

Η πρώτη γραμμή είναι **επικεφαλίδες**, όχι δεδομένα. Το next() την «τραβάει» ώστε ο βρόχος να ξεκινήσει από τη 2η.

DictReader

Χρησιμοποιεί τις επικεφαλίδες ως **κλειδιά**: row["grade"] αντί για row[1]. Πιο ευανάγνωστο — το λεξικό της προηγούμενης φοράς.

Πάλι κείμενο

Το **row["grade"]** είναι **"17"**, όχι 17. Για υπολογισμούς: **float(row["grade"])**.

ΣΥΜΒΟΥΛΉ Για να κρατήσεις τις γραμμές μετά το κλείσιμο του αρχείου: **rows = list(csv.DictReader(f))** μέσα στο with.

pickle: αποθήκευσε ολόκληρη δομή

```
import pickle

phone = {"Άννα": "6971111111", "Νίκος": "6972222222"}

with open("phone.dat", "wb") as f:     # w + b = binary
    pickle.dump(phone, f)

with open("phone.dat", "rb") as f:
    loaded = pickle.load(f)

print(loaded == phone)
True
```

Πότε pickle

Όταν θες να «παγώσεις» μια δομή Python **όπως είναι** — λεξικό με λίστες, λίστα από πλειάδες — και να την ξαναφορτώσεις.

	CSV / TXT	PICKLE
Το διαβάζει άνθρωπος;	ναι	όχι
Ανοίγει σε Excel;	ναι	όχι
Κρατά τύπους;	όχι (όλα str)	ναι
Οποιαδήποτε δομή;	πίνακες	ναι

Δυαδικό αρχείο

Δεν περιέχει κείμενο αλλά **bytes**. Γι' αυτό mode **"wb" / "rb"** και **χωρίς** encoding.

Ασφάλεια

Ποτέ pickle.load σε αρχείο από άγνωστη πηγή.

ΣΤΟ ΕΡΓΑΣΤΉΡΙΟ Θα χρησιμοποιήσουμε **CSV** για τον κατάλογο: μπορείς να το ανοίξεις και να δεις τι αποθηκεύτηκε.

Διαβάζοντας ένα μήνυμα λάθους

```
Traceback (most recent call last):
  File "age.py", line 2, in <module>
    age = int(input("Ηλικία: "))
ValueError: invalid literal for int()
with base 10: 'είκοσι'
```

Από κάτω προς τα πάνω

Η **τελευταία** γραμμή λέει **τι** έγινε (το είδος της εξαίρεσης). Οι από πάνω λένε **πού** (αρχείο, γραμμή).

ΕΞΑΊΡΕΣΗ	ΠΌΤΕ ΣΥΜΒΑΊΝΕΙ	ΠΑΡΆΔΕΙΓΜΑ
`ValueError`	σωστός τύπος, λάθος τιμή	`int("abc")`
`ZeroDivisionError`	διαίρεση με το μηδέν	`10 / 0`
`FileNotFoundError`	το αρχείο δεν υπάρχει	`open("x.txt")`
`KeyError`	το κλειδί δεν υπάρχει	`{}["a"]`
`IndexError`	η θέση δεν υπάρχει	`[1, 2][5]`
`TypeError`	λάθος τύπος	`"5" + 1`

Η ΛΈΞΗ **Εξαίρεση** (exception) = ένα λάθος που συμβαίνει **την ώρα που τρέχει** το πρόγραμμα, όχι λάθος σύνταξης.

Δοκίμασε — κι αν στραβώσει...

```
try:
    age = int(input("Ηλικία: "))
    print("Του χρόνου:", age + 1)
except ValueError:
    print("Δώσε αριθμό, όχι γράμματα!")

print("Τέλος")
```

Πώς δουλεύει

Εκτελούνται οι γραμμές του **try**. Αν μία βγάλει **ValueError**, η Python **πηδάει** αμέσως στο **except** — οι υπόλοιπες γραμμές του try δεν εκτελούνται. Μετά το πρόγραμμα **συνεχίζει** κανονικά.

ΕΊΣΟΔΟΣ	ΕΜΦΑΝΊΖΕΙ	ΤΙ ΈΓΙΝΕ
19	Του χρόνου: 20 · Τέλος	όλο το try, χωρίς except
είκοσι	Δώσε αριθμό, όχι γράμματα! · Τέλος	πήδηξε στο except

Βάλε στο try μόνο ό,τι μπορεί να «σκάσει»

Όσο μικρότερο το try, τόσο πιο καθαρά ξέρεις **ποια** γραμμή έφταιξε.

Πάντα με όνομα

Γράφε **except ValueError:** και όχι σκέτο **except:**. Το σκέτο «καταπίνει» **όλα** τα λάθη — και τα δικά σου.

ΔΙΑΦΟΡΆ ΑΠΌ ΤΟ IF Το if ελέγχει **πριν**. Το try δοκιμάζει και διορθώνει **μετά**. Για «είναι αριθμός;» το try είναι πολύ πιο απλό.

except, else, finally

```
try:
    a = int(input("a: "))
    b = int(input("b: "))
    result = a / b
except ValueError:
    print("Μόνο αριθμοί!")
except ZeroDivisionError:
    print("Διαίρεση με το 0!")
else:
    print("Αποτέλεσμα:", result)
finally:
    print("Τέλος υπολογισμού")
```

ΜΠΛΟΚ	ΕΚΤΕΛΕΊΤΑΙ
`try`	πάντα (μέχρι το πρώτο λάθος)
`except X`	μόνο αν έγινε λάθος **τύπου X**
`else`	μόνο αν **δεν** έγινε κανένα λάθος
`finally`	**πάντα**, ό,τι κι αν έγινε

A, B	ΕΜΦΑΝΊΖΕΙ
`10, 4`	Αποτέλεσμα: 2.5 · Τέλος υπολογισμού
`10, 0`	Διαίρεση με το 0! · Τέλος υπολογισμού
`10, x`	Μόνο αριθμοί! · Τέλος υπολογισμού

Γιατί else κι όχι μέσα στο try

Για να μην «πιάσει» κατά λάθος το except ένα λάθος της print.

ΠΟΎ ΧΡΗΣΙΜΕΎΕΙ ΤΟ FINALLY Για «καθάρισμα» που πρέπει να γίνει πάντα: κλείσιμο αρχείου, μήνυμα «τέλος», αποθήκευση.

Ρώτα ξανά μέχρι να είναι σωστό

```
while True:
    try:
        n = int(input("Ακέραιος: "))
        break
    except ValueError:
        print("Λάθος, ξαναδοκίμασε.")

print("Έδωσες", n)
```

```
Ακέραιος: πέντε
Λάθος, ξαναδοκίμασε.
Ακέραιος: 5.5
Λάθος, ξαναδοκίμασε.
Ακέραιος: 5
Έδωσες 5
```

while True

Ο βρόχος που τρέχει μέχρι να του πεις «φτάνει» — από το 2ο μάθημα.

break μέσα στο try

Φτάνει εκεί **μόνο** αν το int() πέτυχε. Τότε βγαίνουμε από τον βρόχο.

except

Αν απέτυχε, τυπώνει μήνυμα και ο βρόχος **ξαναρωτά**.

Αυτό είναι το πιο χρήσιμο κομμάτι κώδικα της ημέρας

Κάθε φορά που διαβάζεις αριθμό από χρήστη, μπορείς να το αντιγράψεις. Στις 9/11 θα το κάνουμε **συνάρτηση** για να μην το ξαναγράφουμε.

ΠΑΡΑΤΉΡΗΣΗ Το **5.5** απορρίφθηκε γιατί ζητάμε **int**. Για δεκαδικούς άλλαξε σε **float(input(...))**.

Όταν το αρχείο δεν υπάρχει

```
try:
    with open("phone.csv", encoding="utf-8") as f:
        lines = f.readlines()
except FileNotFoundError:
    print("Δεν υπάρχει αρχείο — ξεκινάμε από το μηδέν.")
    lines = []

print(len(lines), "γραμμές")
```

ΚΑΤΆΣΤΑΣΗ	ΑΠΟΤΈΛΕΣΜΑ
δεν υπάρχει το phone.csv	μήνυμα · 0 γραμμές
υπάρχει, με 3 επαφές	3 γραμμές

Το μοτίβο

Δοκίμασε να **φορτώσεις** → αν λείπει, ξεκίνα με **κενή** δομή → στο τέλος **αποθήκευσε**. Αυτό θα κάνουμε στον κατάλογο.

Η πρώτη εκτέλεση

Την **πρώτη** φορά που τρέχει ο κατάλογος, το phone.csv **δεν έχει φτιαχτεί ακόμα**. Αυτό δεν είναι λάθος — είναι αναμενόμενο.

Άλλα σφάλματα αρχείων

PermissionError: δεν έχεις δικαίωμα (π.χ. το αρχείο είναι ανοιχτό στο Excel).

ΣΥΜΒΟΥΛΉ Αν το πρόγραμμα «δεν βρίσκει» ένα αρχείο που βλέπεις, σχεδόν πάντα τρέχει από **άλλον φάκελο**. Στο VS Code: Open Folder.

Σηκώνουμε εμείς μια εξαίρεση

```
try:
    grade = float(input("Βαθμός: "))
    if not 0 <= grade <= 20:
        raise ValueError("εκτός ορίων 0-20")
except ValueError as e:
    print("Λάθος:", e)
else:
    print("Εντάξει:", grade)
```

ΕΙΣΟΔΟΣ	ΕΜΦΑΝΙΖΕΙ
17	Εντάξει: 17.0
25	Λάθος: εκτός ορίων 0-20
abc	Λάθος: could not convert string to float: 'abc'

raise

Το **25** είναι έγκυρος αριθμός, αλλά **όχι έγκυρος βαθμός**. Με **raise** το κάνουμε κι αυτό ValueError — και το πιάνει **το ίδιο** except.

except ... as e

Το **e** κρατά το **μήνυμα** της εξαίρεσης. Το τυπώνουμε για να ξέρει ο χρήστης τι έφταιξε.

Ένα σημείο για όλα τα λάθη

Είτε γράψει γράμματα είτε αριθμό εκτός ορίων, ο χρήστης παίρνει μήνυμα **από το ίδιο except**. Ο κώδικας μένει απλός.

ΜΠΡΟΣΤΆ Το raise θα το χρησιμοποιούμε πολύ στις συναρτήσεις (9/11) και στις κλάσεις (30/11): «αυτή η τιμή δεν επιτρέπεται».

Ποια γράμματα θα τυπωθούν;

ΕΚΦΏΝΗΣΗ

Για καθεμία από τις τρεις εισόδους του πίνακα, γράψε **με τη σειρά** τα γράμματα που εμφανίζονται.

```
try:
    x = int(input())
    print("Α")
    y = 10 / x
    print("Β")
except ValueError:
    print("Γ")
except ZeroDivisionError:
    print("Δ")
else:
    print("Ε")
finally:
    print("Ζ")
```

ΛΎΣΗ

ΕΊΣΟΔΟΣ	ΤΥΠΏΝΕΙ	ΓΙΑΤΊ
5	**Α Β Ε Ζ**	κανένα λάθος → else
0	**Α Δ Ζ**	λάθος στη διαίρεση: το Β χάνεται
abc	**Γ Ζ**	λάθος στο int(): ούτε Α

Το Ζ σε όλες

Το **finally** εκτελείται **πάντα** — είτε έγινε λάθος είτε όχι.

Το Ε μόνο μία φορά

Το **else** μόνο όταν **δεν** έγινε κανένα λάθος.

Η ΟΥΣΊΑ Μόλις γίνει λάθος, η εκτέλεση του try **σταματά εκεί**. Ό,τι ακολουθεί μέσα στο try δεν εκτελείται ποτέ.

Ασφαλής είσοδος βαθμών

ΕΚΦΏΝΗΣΗ

Διάβασε **5 βαθμούς** (0–20). Αν ο χρήστης γράψει κάτι που δεν είναι αριθμός ή είναι εκτός ορίων, εμφάνισε μήνυμα και **ξαναρώτα** τον ίδιο βαθμό. Στο τέλος εμφάνισε τον μέσο όρο.

ΛΎΣΗ

```
grades = []
while len(grades) < 5:
    try:
        g = float(input(f"Βαθμός {len(grades) + 1}: "))
        if not 0 <= g <= 20:
            raise ValueError("εκτός ορίων 0-20")
        grades.append(g)
    except ValueError as e:
        print("Λάθος:", e)
print(f"Μέσος όρος: {sum(grades) / len(grades):.2f}")
```

```
Βαθμός 1: 15
Βαθμός 2: abc
Λάθος: could not convert ...
Βαθμός 2: 25
Λάθος: εκτός ορίων 0-20
Βαθμός 2: 12
...
Μέσος όρος: 14.90
```

Γιατί while len(grades) < 5

Μετράμε **επιτυχίες**, όχι προσπάθειες. Με for θα χάναμε βαθμούς σε κάθε λάθος.

ΤΑ ΠΆΝΤΑ ΜΑΖΊ while, try/except, raise, λίστα, f-string. Δοκίμασε και κενή είσοδο (σκέτο Enter): πιάνεται κι αυτή από το ίδιο except.

Στατιστικά από αρχείο CSV

```
import csv

try:
    with open("grades.csv", newline="",
              encoding="utf-8") as f:
        rows = list(csv.DictReader(f))
except FileNotFoundError:
    print("Δεν βρέθηκε το grades.csv")
else:
    grades = [float(r["grade"]) for r in rows]
    passed = [r["name"] for r in rows
              if float(r["grade"]) >= 10]
    print(f"Μαθητές: {len(rows)}")
    print(f"Μέσος όρος: {sum(grades) / len(grades):.2f}")
    print("Πέρασαν:", ", ".join(passed))
```

ΕΚΦΏΝΗΣΗ

Διάβασε το **grades.csv** και εμφάνισε πλήθος μαθητών, **μέσο όρο** και **ποιοι πέρασαν**. Αν το αρχείο λείπει, εμφάνισε μήνυμα αντί για σφάλμα.

GRADES.CSV

```
name,grade
Άννα,17
Νίκος,9
Μαρία,18.5
Γιώργος,11
Ελένη,7.5
```

```
Μαθητές: 5
Μέσος όρος:
  12.60
Πέρασαν: Άννα,
 Μαρία, Γιώργος
```

", ".join(λίστα)

Ενώνει τα στοιχεία μιας λίστας κειμένων με **", "** ανάμεσα: ["Άννα", "Μαρία"] → "Άννα, Μαρία".

Το αρχείο είναι έτοιμο

Το **grades.csv** είναι στον φάκελο του μαθήματος. Άνοιξέ το και στο Excel για να δεις τον πίνακα.

ΔΟΚΊΜΑΣΕ Μετονόμασε το αρχείο και τρέξε ξανά: πρέπει να δεις το μήνυμα, όχι Traceback. Μετά πρόσθεσε μια γραμμή στο CSV με το Excel.

Κατάλογος που θυμάται

```
import csv
FILE = "phone.csv"

phone = {}
try:
    with open(FILE, newline="", encoding="utf-8") as f:
        for name, number in csv.reader(f):
            phone[name] = number
except FileNotFoundError:
    pass                            # 1η φορά: κενός κατάλογος

while True:
    choice = input("\n1.Προσθήκη 2.Αναζήτηση 3.Όλοι 0.Έξοδος: ")
    if choice == "1":
        name = input("Όνομα: ")
        phone[name] = input("Τηλέφωνο: ")
    elif choice == "2":
        name = input("Όνομα: ")
        print(phone.get(name, "Δεν βρέθηκε"))
    elif choice == "3":
        for name in sorted(phone):
            print(f"{name}: {phone[name]}")
    elif choice == "0":
        with open(FILE, "w", newline="", encoding="utf-8") as f:
            csv.writer(f).writerows(phone.items())
        print(f"Αποθηκεύτηκαν {len(phone)} επαφές.")
        break
    else:
        print("Λάθος επιλογή")
```

ΕΚΦΏΝΗΣΗ

Ο κατάλογος της προηγούμενης φοράς, που **φορτώνει** τις επαφές στην αρχή και τις **αποθηκεύει** στην έξοδο.

1 · Φόρτωση

Αν υπάρχει το phone.csv, γεμίζει το λεξικό. Αν όχι, το **pass** λέει «τίποτα, συνέχισε».

2 · Μενού

Ίδιο με τις 26/10 — με λεξικό.

3 · Αποθήκευση

items() δίνει ζευγάρια → **writerows**.

ΔΟΚΊΜΑΣΕ Πρόσθεσε 2 επαφές, βγες με 0, **ξανατρέξε** το πρόγραμμα και πάτα 3. Οι επαφές είναι εκεί! Άνοιξε και το phone.csv να τις δεις.

Τι κρατάμε

Αρχεία

with open(όνομα, mode, encoding="utf-8"). **r** διαβάζει, **w** σβήνει και γράφει, **a** προσθέτει. Ό,τι διαβάζεις είναι str.

CSV & pickle

csv.writer / DictReader για πίνακες που ανοίγουν και στο Excel. **pickle** για ολόκληρες δομές Python.

Εξαιρέσεις

try δοκιμάζει, **except** πιάνει, **else** αν όλα καλά, **finally** πάντα. **raise** για δικά σου λάθη.

Το πρόγραμμά σου πλέον **θυμάται** (αρχεία) και **αντέχει** (εξαιρέσεις). Αυτά τα δύο ξεχωρίζουν ένα «πρόγραμμα άσκησης» από ένα πρόγραμμα που μπορεί να χρησιμοποιήσει κάποιος άλλος.

Επόμενη φορά · Δευ 9/11

Συναρτήσεις: το «ρώτα ξανά μέχρι να είναι σωστό» γίνεται μία γραμμή — **read_int("Ηλικία: ")** — που καλείς όσες φορές θες.

ΓΙΑ ΤΟ ΣΠΊΤΙ Ημερολόγιο: ρώτα μια σημείωση, **πρόσθεσέ** τη (mode "a") στο notes.txt, και στο τέλος τύπωσε όλο το αρχείο αριθμημένο.